



WeCloudData

Pandas

Introduction





Agenda

- Introduction
- Pandas Series
- DataFrame





WeCloudData

Pandas Introduction



What is Pandas

Pandas

Pandas is an open-source Python library used for:

- Organizing data
- Cleaning data
- Exploring data
- Analyzing data
- Preparing data for machine learning



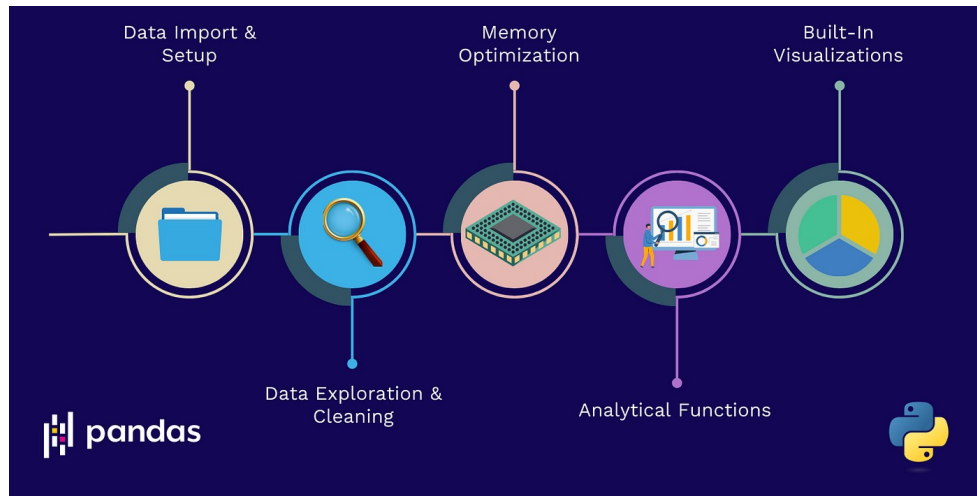


Who Use Pandas

Pandas

Pandas is used by:

- Data Analysts
- Data Scientists
- Business Analysts
- Researchers
- Machine Learning Engineers



[Source](#)



Why not just use Python lists?

Why Use Pandas?

Imagine you are managing data for **10,000 students**.

You now have:

- 10,000 names
- 10,000 ages
- 10,000 cities

All stored separately.

Problem

What happens if:

- A student is removed?
- A value is inserted incorrectly?
- One list becomes shorter?

Everything becomes misaligned.

```
names = [...]  
ages = [...]  
cities = [...]
```

As datasets grow, managing separate lists becomes difficult and error-prone.



The Pandas solution: one structured table

Why Use Pandas?

Imagine receiving student data:

Name	Age	City
Abdullah	22	Riyadh
Sara	24	Jeddah
Ahmed	21	Dammam



The Pandas solution: one structured table

Why Use Pandas?

With Pandas:

```
df = pd.DataFrame({  
    "Name": [...],  
    "Age": [...],  
    "City": [...]  
})
```

Everything stays together:

	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam





Two core structures

Data Structure

Pandas provides two fundamental structures:

DataFrame

ID	Name	Age	City
S1	Abdullah	22	Riyadh
S2	Sara	24	Jeddah
S3	Ahmed	21	Dammam

Full table: multiple related columns

Series

index	Age
S1	22
S2	24
S3	21

One labeled column



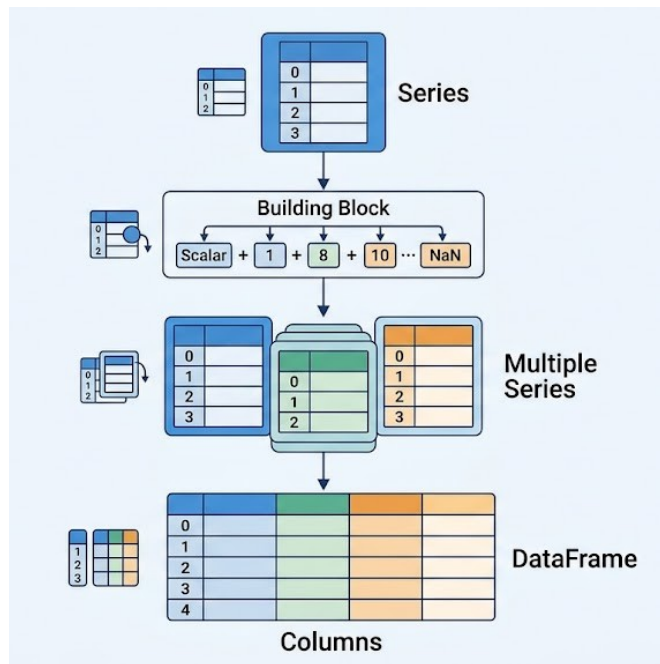
WeCloudData

Pandas Series



Relation between Series and DataFrame

Pandas Series Intro



A DataFrame is essentially a collection of Series.

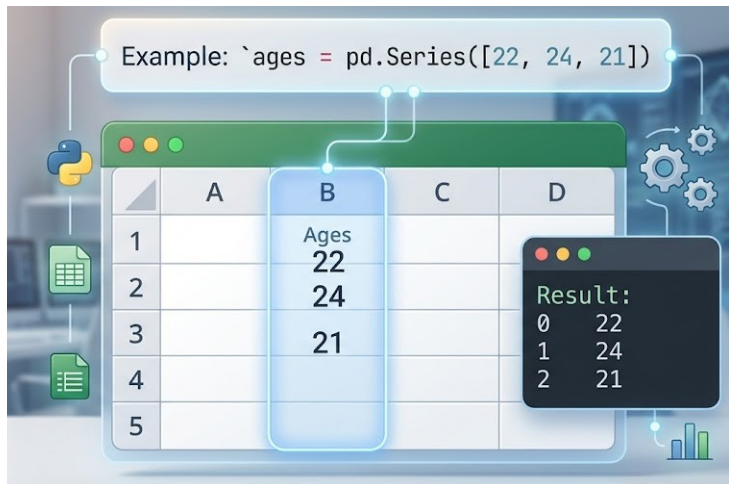


What is Series

Pandas Series: What

A series is:

A one-dimensional labeled array capable of holding any data type.





Every Series Has an Index and Values

Pandas Series: What

Let's Define a series:

```
ages = pd.Series([22, 24, 21])
```

```
ages.index  
# RangeIndex(start=0, stop=3, step=1)
```

Default index

```
ages.values  
# array([22, 24, 21])
```

Stored data values



Series with Custom Labels

Pandas Series: Custom Labels

Custom labels make values easier to identify.

```
ages = pd.Series(  
    [22, 24, 21],  
    index=["Abdullah", "Sara", "Ahmed"]  
)
```

```
ages["Sara"]  
# return 24
```

Index	Age
Abdullah	22
Sara	24
Ahmed	21



When Should You Use a Series?

Pandas Series: When

Use a Series when you are focusing on one column, feature, or measurement.

Analyze one column

Age, score, salary, price

Calculate statistics

mean(), max(), min()

Create filters

`df["Age"] > 21`

Keep row labels

values stay connected to their index



Simple Series

Pandas Series: Example

Operations

```
ages.mean()  
# 22.33
```

```
ages.max()  
# 24
```

```
ages > 21  
# Abdullah  True  
# Sara      True  
# Ahmed    False
```

These same ideas will appear again when we filter DataFrames.



WeCloudData

Pandas DataFrame



From Series to DataFrame

One Series is useful. Real datasets usually have multiple related Series.

```
names = pd.Series(["Abdullah", "Sara", "Ahmed"])
ages = pd.Series([22, 24, 21])
cities = pd.Series(["Riyadh", "Jeddah", "Dammam"])
```

A DataFrame brings related Series together into one table.

	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam



DataFrame

Pandas DataFrame: What

A dataframe is:

A two-dimensional tabular structure consisting of rows and columns.

Think Of It As

- Excel Spreadsheet
- SQL Table
- Student Records

index	columns		
	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam



Creating a DataFrame

Pandas DataFrame: Create

```
df = pd.DataFrame({  
    "Name": ["Abdullah", "Sara", "Ahmed"],  
    "Age": [22, 24, 21],  
    "City": ["Riyadh", "Jeddah", "Dammam"]  
})
```

Result

A structured table ready for analysis

	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam



Why Use a DataFrame?

Pandas DataFrame: Why

It keeps related data together and makes analysis easier.

Separate lists

Easy to misalign
Harder to filter
Harder to summarize
Harder to inspect

DataFrame

One structured table
Rows stay together
Columns have names
Built-in analysis tools



**Trying to do data analysis
using only nested lists
and nested loops.**



**Using Pandas DataFrames
for everything.**

-  **Fast lookup**
-  **Filtering**
-  **Data Alignment**
-  **Time-series analysis**



When Should You Use a Pandas DataFrame? When

Use DataFrame when your data has multiple related columns.

CSV / Excel files

Rows and columns of records

Student or customer data

Multiple attributes per person

Filtering rows

Find records that match a condition

Summarizing tables

Calculate averages, counts, and totals

Most Pandas workflows start by loading data into a DataFrame.



Simple DataFrame

Pandas DataFrame: Example Statistics

Pandas can answer table questions with short commands.

	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam

```
df["Age"].mean()  
# 22.33
```

```
df["Age"].max()  
# 24
```

```
df["City"].count()  
# 3
```

```
df["Age"].min()  
# 21
```



Indexes

DataFrame

Indexes identify rows.

```
df.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

Custom Index:

```
df.index = ["S1", "S2", "S3"]
```

	Name	Age	City
S1	Abdullah	22	Riyadh
S2	Sara	24	Jeddah
S3	Ahmed	21	Dammam



WeCloudData

Pandas Index



What Is an Index?

Pandas: Index

An index identifies rows or values. Pandas creates one automatically if we do not provide it.

	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam

Default index:

0, 1, 2, ...

Created automatically.

Custom index:

S1, S2, S3

Chosen by us when labels are useful.



The Index Is Also an Object

Pandas: Index

Pandas stores row labels in a separate Index object.

```
obj = pd.Series(range(3), index=["a", "b", "c"])  
  
obj.index  
# Index(["a", "b", "c"], dtype="object")
```

This means the row labels are stored separately from the values.

```
obj.index[1:]  
# Index(["b", "c"], dtype="object")
```

You can view and slice an Index just like a sequence.



Default Index vs Custom Index

Pandas: Index

Custom labels can make row access easier to understand.

```
df.index  
# RangeIndex(start=0, stop=3, step=1)
```

Index	Name	Age	City
0	Abdullah	22	Riyadh
1	Sara	24	Jeddah
2	Ahmed	21	Dammam

```
df.index = ["S1", "S2", "S3"]
```

Index	Name	Age	City
S1	Abdullah	22	Riyadh
S2	Sara	24	Jeddah
S3	Ahmed	21	Dammam



Index Labels Are Pandas: Index Immutable

You can replace the whole index, but you cannot edit one label in place.

```
index = pd.Index(["a", "b", "c"])  
  
index[1:]  
# Index(["b", "c"], dtype="object")
```

```
index[1] = "d"  
# TypeError
```

Allowed: viewing and slicing the Index.

Not allowed: changing one label directly.

This rule keeps index labels stable and prevents accidental one-label changes.



Can Index Labels Repeat?

Pandas: Index

Pandas allows duplicate labels by default, but duplicates can make selection harder to interpret.

Index	Col1	Col2
11	D1	D1
12	D2	D2
12	D3	D3
14	D4	D4

```
df.index.duplicated()  
# array([False, False, True, False])
```

```
df.index.is_unique  
# False
```

Duplicate index labels are not always wrong, but they require extra care.



Why Duplicate Labels

Pandas Index
Matter

Duplicate labels often show up when **combining or appending** datasets.

```
df2 = pd.DataFrame({"Col1": ["D5"], "Col2": ["D5"]}, index=[12])  
pd.concat([df, df2])
```

Pandas allows duplicate labels by default, but duplicates can make results harder to interpret.

Index	Col1	Col2
11	D1	D1
12	D2	D2
13	D3	D3
14	D4	D4
12	D5	D5

Key idea: check uniqueness before relying on labels.



Creating an Index from Columns

Pandas: Index

`set_index()` turns one or more existing columns into the index.

Index x	Col1	Col2	Col3	Col4
0	D11	D12	D13	D14
1	D11	D13	D23	D24
2	D31	D32	D33	D34

Before: Col1 and Col2 are regular columns.

```
df = df.set_index(["Col1", "Col2"])
```

Index		Col3	Col4
Col1	Col2		
D11	D12	D13	D14
D11	D13	D23	D24
D31	D32	D33	D34

After: Col1 and Col2 become index levels.



MultIndex: More Than One Index

Pandas: Index Level

When multiple columns become the index, Pandas creates a hierarchical index.

Index		Col3	Col4
Col1	Col2		
D11	D12	D13	D14
D11	D13	D23	D24
D31	D32	D33	D34

Col1 + Col2
act together as row labels.

```
df.index  
# MultiIndex([("D11", "D12"),  
#            ("D11", "D13"), ...])
```

A MultiIndex is useful when one label is not enough to identify each row.



loc vs iloc

DataFrame

.loc

Labels

```
df.loc['row_name']
```

```
df.loc[condition]
```

.iloc

Positions

```
df.iloc[0]
```

```
df.iloc[:5]
```

pandas DataFrame Indexing Methods

[Source: medium article](#)



loc vs iloc

DataFrame

Access by Label

```
df.loc["S1"]
```

S1	
Name	Abdullah
Age	22
City	Riyadh

Access by Position

```
df.iloc[0]
```

S1	
Name	Abdullah
Age	22
City	Riyadh

	Name	Age	City
S1	Abdullah	22	Riyadh
S2	Sara	24	Jeddah
S3	Ahmed	21	Dammam



Summary

Pandas

A Python library for working with structured data.

Series

- One-dimensional
- Single column
- Values + Index

DataFrame

- Two-dimensional
- Multiple columns
- Collection of Series



WeCloudData

Thank You

We'd love to hear from you. If you have any questions,
feel free to reach out to our team anytime!